

Internet Technology Lab Report

Assignment – 1

Problem Statement:

Implement a TCP-based key-value store where the server maintains separate data stores for different clients based on their IP address.

Student Details

- **Name:** Sumesh Ranjan Majee
 - **Roll Number:** 002310501020
 - **Year:** 3rd Year B.C.S.E
 - **Group:** A1
 - **Date:** 11th February 2026
-

Subject:

Internet Technology

1. Assignment Statement

Problem: Implement a TCP-based key-value store where the server maintains separate data stores for different clients based on their IP address. The system must support get and put operations for key-value pairs stored in memory. Additionally, the system must implement an authorization mechanism where a "guest" user can upgrade to a "manager" role to access key-value stores of other clients.

The assignment requires two implementations:

1. **Command Line Interface (CLI):** Using Python socket programming.
2. **Web Application:** Using Node.js with a web browser client.

2. Design

2.1 Purpose

The purpose of this program is to demonstrate network programming concepts, specifically Client-Server architecture. It explores state management (in-memory storage), concurrency (handling multiple clients), and role-based access control (RBAC).

2.2 System Architecture

Version A: Python TCP Socket Architecture

- **Server:** A multi-threaded TCP server. The main thread listens on port 4000. Upon accepting a connection, a new thread is spawned to handle that specific client connection.
- **Storage:** A global dictionary `DATA_STORE` structured as `{ 'client_ip': { 'key': 'value' } }`. Access to this shared resource is synchronized using a `threading.Lock`.
- **Client:** A script that parses command-line arguments, connects to the server, sends commands, and prints responses.

Version B: Node.js Web Architecture

- **Server:** An Express.js server providing a REST API (/api/put, /api/get, /api/auth). It serves a frontend using EJS templates.
- **Storage:** A global JavaScript Object dataStore serves as the in-memory database.
- **Client:** A Browser-based UI (EJS/HTML) that uses the Fetch API to communicate with the backend.

2.3 Input and Output Format

CLI Version:

- **Input:** Command line arguments. Example: ./client localhost 4000 put city Kolkata.
- **Output:** Server response strings printed to the console (e.g., "OK", "Kolkata", "<blank>").

Web Version:

- **Input:** JSON payloads sent via HTTP POST (e.g., { "key": "city", "value": "Kolkata" }) or Query parameters for GET requests.
- **Output:** JSON responses containing status messages or retrieved values.

3. Implementation

3.1 Python Implementation (Socket Programming)

The Python server uses socket for networking and threading for concurrency.

Key Data Structure:

Python

```
# Structure: { 'client_ip': { 'key': 'value' } }
DATA_STORE = {}
store_lock = threading.Lock()
```

Request Handling (Server):

The handle_client function processes the input stream. It differentiates between guest and manager roles during a GET request.

Python

```
# KVServer.py Snippet
if ":" in key and role == "manager":
    target_ip, target_key = key.split(":", 1)
    with store_lock:
        target_store = DATA_STORE.get(target_ip, {})
        val = target_store.get(target_key)
else:
    my_store = get_client_store(client_ip)
    val = my_store.get(key)
```

Client Logic:

The client parses variable arguments to allow chaining commands.

Python

```
# client.py Snippet
while i < len(args):
    cmd = args[i]
    if cmd.lower() == "put":
        # ... constructs message ...
        sock.sendall(msg.encode())
    elif cmd.lower() == "get":
        # ... sends get request ...
```

3.2 Node.js Implementation (Web Application)

The Node.js server uses express and body-parser. It mimics the logic of the Python server but uses HTTP headers for authorization tokens.

API Endpoint for GET:

JavaScript

```
// server.js Snippet
app.get("/api/get", (req, res) => {
  const { key, targetIp } = req.query;
  const clientToken = req.headers["x-auth-token"];

  if (targetIp) {
    if (clientToken === ADMIN_TOKEN) {
      // Allow access to other IP's store
      storeToRead = getStore(targetIp);
    } else {
      return res.status(403).json({ value: "ACCESS DENIED..." });
    }
  }
  // ...
});
```

4. Test Cases & Results

To verify correctness, the following test cases were executed.

4.1 Test Case 1: Basic Storage (Guest)

- **Objective:** Verify a client can store and retrieve its own data.
- **Input:** python client.py localhost 4000 put city Kolkata get city
- **Expected Result:** "Kolkata"
- **Actual Result:** "Kolkata" (Status: **PASS**)

4.2 Test Case 2: Data Isolation

- **Objective:** Verify Client A cannot access Client B's data by default.
- **Setup:** 1. Client A (192.168.1.10) runs put secret treasure.
2. Client B (192.168.1.20) runs get secret.
- **Expected Result:** Client B receives <blank> (or empty), as the key does not exist in B's store.
- **Actual Result:** <blank> (Status: **PASS**)

4.3 Test Case 3: Authorization & Role Upgrade

- **Objective:** Verify a user can upgrade to manager using the password "admin123".
- **Input:** python client.py localhost 4000 auth admin123
- **Expected Result:** "ROLE_UPDATED: You are now a Manager"
- **Actual Result:** "ROLE_UPDATED: You are now a Manager" (Status: **PASS**)

4.4 Test Case 4: Manager Access (Cross-Client Access)

- **Objective:** Verify a Manager can access another client's data.
- **Input:** python client.py localhost 4000 auth admin123 get 127.0.0.1:city
- **Note:** Assumes "city" was set to "Kolkata" by localhost previously.
- **Expected Result:** "Kolkata"
- **Actual Result:** "Kolkata" (Status: **PASS**)

4.5 Test Case 5: Web Interface (Node.js)

- **Objective:** Verify the Web UI interacts correctly with the API.
- **Action:** 1. Open browser to http://localhost:3000.
2. Enter Key: Frame, Value: React -> Click "Put Data".
3. Enter Key: Frame -> Click "Get Data".
- **Expected Result:** Activity Log shows PUT Frame=React : OK and GET (Self) Frame : React.
- **Actual Result:** Log updated correctly. (Status: **PASS**)

5. Analysis

Discussion on Results: The system successfully implements the requirements. Both the Python and Node.js versions maintain data persistence in memory for the duration of the server runtime.

Correctness & Concurrency:

- **Python:** The use of threading.Lock() in KVServer.py ensures thread safety. Without this, simultaneous writes from different threads could corrupt the DATA_STORE dictionary.
- **Node.js:** Node.js runs on a single-threaded event loop. While this avoids traditional thread-locking issues for simple dictionary assignments, the logic relies on the synchronous execution of the callback functions to ensure data consistency.

Known Limitations & Improvements:

1. **Volatile Storage:** As specified in the assignment, data is stored in memory. If the server crashes, all data is lost. *Improvement:* Implement file-based persistence (JSON/SQLite).
2. **Security:** The password "admin123" is hardcoded and sent in plain text. *Improvement:* Use hashing and environment variables.
3. **Network Protocol:** The Python client creates a new socket connection for every execution. *Improvement:* Implement a persistent interactive shell to keep the connection open.

6. Comments

Evaluation:

This assignment provided practical experience in implementing Application Layer protocols. The contrast between the persistent TCP connection model (Python) and the stateless HTTP model (Node.js) highlighted different approaches to session management.

Learning Outcomes:

1. Understood how to manage multiple client states using IP addresses as unique identifiers.
2. Learned to implement manual Application Layer protocols (defining message formats like "put key value").
3. Gained experience in handling Cross-Origin Resource Sharing (CORS) and API token management in the Node.js implementation.

Complexity:

The assignment was of moderate difficulty. The hardest part was ensuring the "Manager" logic correctly parsed the target_ip:key format in Python and managing the authentication tokens in the stateless web environment.